

TABLE OF CONTENTS

S.No	Experiment Title	Date	Sign
1.	Write a code/program in OpenCV(Using Python) to read , resize and display an image.	03/07/26	
2.	Write a program in OpenCV(Using Python) to show multiple images in a single frame.	10/07/26	
3.	Write a program in OpenCV(Using Python) crop an image also show its original image.	17/07/26	
4.	Write a program in OpenCV using python to detect edges of an image.	24/07/26	
5.	Write a program in OpenCV for detecting a face by using AI code.	29/07/26	
6.	Write a program in OpenCV for detecting eyes on face by using AI code.	05/08/26	
7.	Write a program in OpenCV for erosion and dilation of an image.	12/08/26	
8.	Write a program of computer vision to detect multiple faces.	19/08/26	
9.	Write a program in computer vision using tesseract for text detection and print it.	02/09/26	
10.	Write a program in computer vision for real time object detection from a video using YOLO.	09/09/26	
11.	Write a program in computer vision for using gushing blur function.	16/09/26	
12.	Write a Python program using OpenCV to convert an input image into grayscale and apply the Laplacian filter to detect edges. Display both the original image and the resulting Laplacian image.	23/09/26	
13.	Write a Python program using OpenCV to increase and decrease the brightness of an input image. Display the original, brightened, and darkened image (Image Segmentation).	23/09/26	
14.	Write a program in computer vision for live streaming or opening a camera.	02/10/26	
15.	Write a program in computer vision for contour detection and masking.	02/10/26	

Experiment : Write a code/program in OpenCV(Using Python) to read , resize and display an image.

Aim: Using OpenCV library of python , insert an image and display it.

Algorithm/ Procedure:

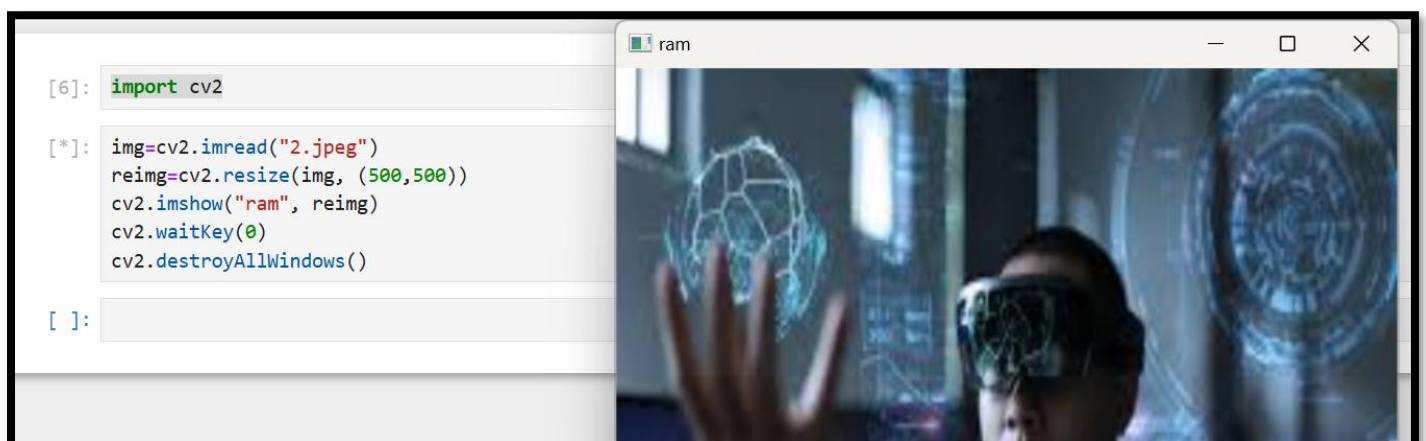
1. Start Program by opening Jupiter Navigator
2. Open Jupiter Notepad
3. Import CV2 library
4. Insert path of an image to read.
5. Put resize function for resizing image
6. Show image
7. Exit

Program:

```
import cv2
img=cv2.imread("1.jpg")
reimg=cv2.resize(img, (500,500)) #used function for resizing its dimension
cv2.imshow("ram", reimg)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Input/ Output

Input an image after the program executed , image displayed into a resized form



Result: Program successfully executed, Image successfully resized and displayed.

Experiment -2

Date:10/07/26.

Experiment : Write a program in OpenCV(Using Python) to show multiple images in a single frame.

Aim: Using OpenCV and python, show multiple images into a single frame.

Algorithm/ Procedure:

1. Start Program by opening Jupiter Navigator
2. Open Jupiter Notepad
3. Import CV2 library
4. Import numpy variable
5. Insert path of an image to read.
6. Put resize function for resizing image
7. Adjust images horizontally and vertically
8. Use waitKey() function for exit
9. Exit

Program:

```
import cv2
import numpy as np
img=cv2.imread("SRM.png")
reimg=cv2.resize(img, (200, 200))
```

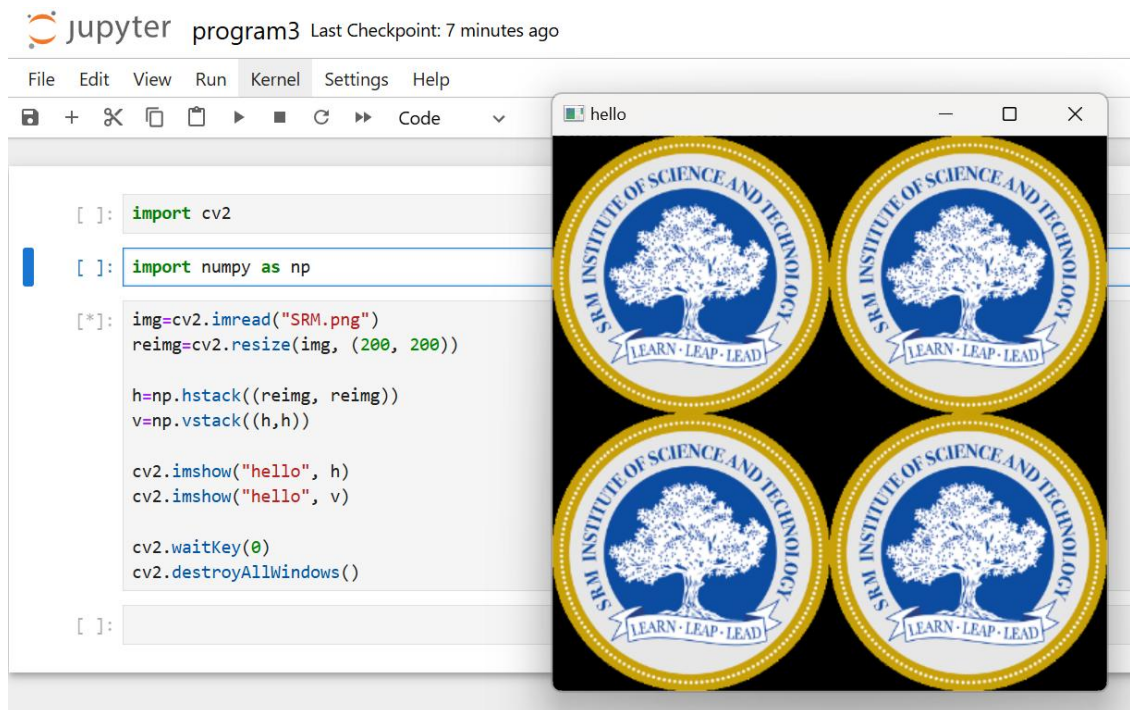
```
h=np.hstack((reimg, reimg))
v=np.vstack((h,h))
```

```
cv2.imshow("hello", h)
cv2.imshow("hello", v)
```

```
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Input/Output:

Input a figure (SRM Seal) and adjust it into frame 4x4



Result: Program successfully executed, image successfully framed.

Experiment -3

Date:17/07/26.

Experiment : Write a program in OpenCV(Using Python) crop an image also show its original image.

Aim: Using OpenCV and python, crop image from original image.

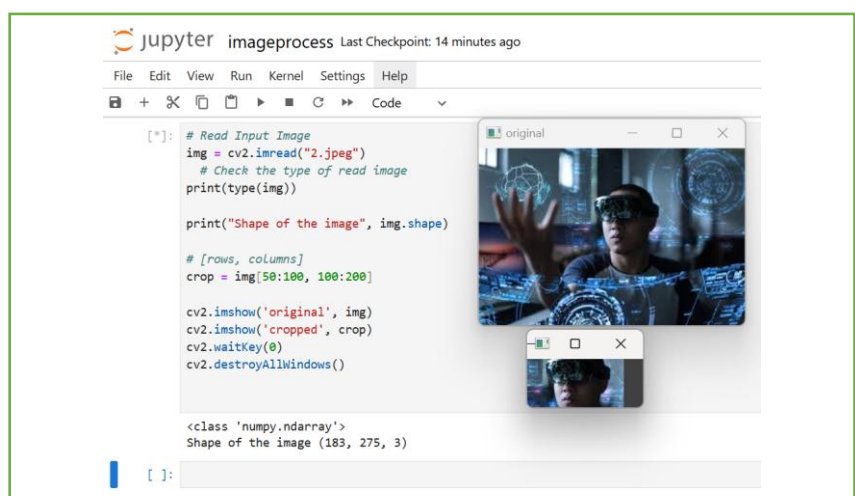
Algorithm/ Procedure:

1. Start Program by opening Jupiter Navigator
2. Open Jupiter Notepad
3. Import CV2 library
4. Print type of image
5. Print image original size
6. Crop image by using crop variable and ratio
7. Use waitKey() function for exit
8. Exit

Program:

```
Import cv2
# Read Input Image
img = cv2.imread("2.jpeg")
# Check the type of read image
print(type(img))
print("Shape of the image", img.shape)
# [rows, columns]
crop = img[50:180, 100:300]
cv2.imshow('original', img)
cv2.imshow('cropped', crop)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Input/output



Result: Image cropped successfully

Experiment -4

Date:24/07/26.

Experiment : Write a program in OpenCV using python to detect edges of an image.

Aim: Using *OpenCV* to find the edges of an image.

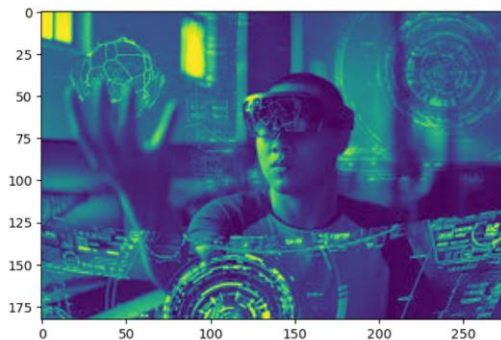
Algorithm/ Procedure:

1. Start Program by opening Jupiter Navigator
2. Import cv2
3. Read image
4. Use canny function
5. Result image
6. Exit

Program:

```
import cv2
import matplotlib.pyplot as plt
img=cv2.imread('ss.jpeg', 0)
plt.imshow(img)
```

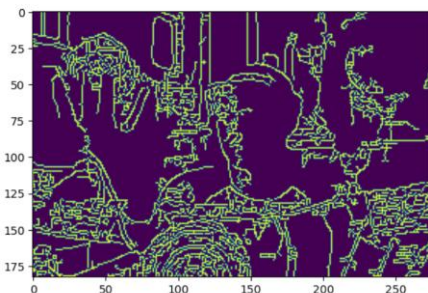
Input of image with name ss.jpg



```
img1=cv2.Canny(img,10,400)
plt.imshow(img1)
```

Output:

<matplotlib.image.AxesImage at 0x1b5b7e2c150>



Result: Edges detected successfully

Experiment : Write a program in OpenCV for detecting a face by using AI code.

Aim: *Using OpenCV* to detect a face using AI cascade.

Procedure:

1. Start Jupyter Notebook
2. Import CV2
3. Import image that is required to detect.
4. Convert image into grayscale
5. Import cascade XML cascade file for detect face.
6. Use for loop for creating square on face
7. Display image
8. End program

Program/ Code :

```
import cv2
import matplotlib.pyplot as plt

# Load the cascade
face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades + 'haarcascade_frontalface_default.xml')

# Read the input image
img = cv2.imread('aa.jpg')
img1=cv2.resize(img, (200,200))

# Convert into grayscale
gray = cv2.cvtColor(img1, cv2.COLOR_BGR2GRAY)

# Detect faces
faces = face_cascade.detectMultiScale(img1, 1.2, 5)

# Draw rectangle around the faces
for (x, y, w, h) in faces:
    cv2.rectangle(img1, (x, y), (x + w, y + h), (255, 0, 0), 2)

# Display the output
cv2.imshow('img', img1)
cv2.waitKey(0)
```

Input / Output

Input an image with name aa.jpg

Output:

```
# Load the cascade
face_cascade = cv2.CascadeClassifier(cv2.data.harcascades + 'haarcascade_frontalface_default.xml')

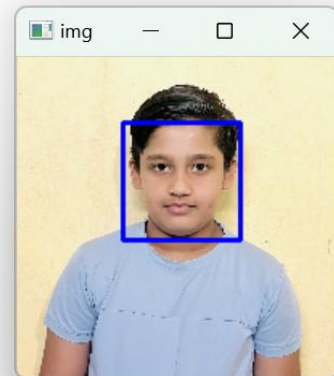
# Read the input image
img = cv2.imread('aa.jpg')
img1=cv2.resize(img, (200,200))

# Convert into grayscale
gray = cv2.cvtColor(img1, cv2.COLOR_BGR2GRAY)

# Detect faces
faces = face_cascade.detectMultiScale(img1, 1.2, 5)

# Draw rectangle around the faces
for (x, y, w, h) in faces:
    cv2.rectangle(img1, (x, y), (x + w, y + h), (255, 0, 0), 2)

# Display the output
cv2.imshow('img', img1)
cv2.waitKey(0)
```



Result:

Face detected successfully using opencv programming.

Experiment : Write a program in OpenCV for detecting eyes on face by using AI code.

Aim: Using OpenCV to detect eyes using AI cascade.

Procedure:

1. Start Jupyter Notebook
2. Import CV2
3. Import image that is required to detect.
4. Convert image into grayscale
5. Import cascade XML cascade file for detect face and eyes.
6. Use for loop for creating square on eyes
7. Display image
8. End program

Program/ Code :

```
import cv2

eye_cascade = cv2.CascadeClassifier('.\Data\haarcascade_eye.xml')
nadia = cv2.imread('./Data/Nadia_Murad.jpg',0)

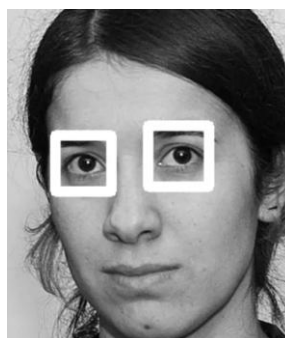
def detect_eyes(img):
    face_img = img.copy()
    eyes = eye_cascade.detectMultiScale(face_img,scaleFactor=1.2,minNeighbors=5)

    for (x,y,w,h) in eyes:
        cv2.rectangle(face_img, (x,y), (x+w,y+h), (255,255,255), 10)
    return face_img

result = detect_eyes(nadia)

while True:
    cv2.imshow(",result)
    code = cv2.waitKey(10)
    if code == ord('q'):
        break
```

Input / Output



Result : Eyes detected successfully by using OpenCV

Experiment -7

Experiment : Write a program in OpenCV for erosion and dilation of an image.

Aim: *Using OpenCV* erosion and dilation of an image.

Procedure:

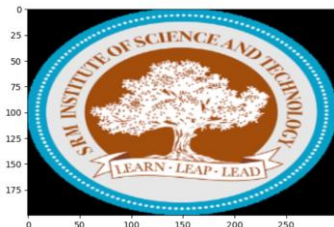
1. Start Jupyter Notebook
2. Import CV2
3. Import image that is required to erosion and dilation.
4. Resize image
5. Use erode function for filling pixels
6. Show image
7. Dilate image or remove pixels.
8. Display image
9. End program

Program/ Code :

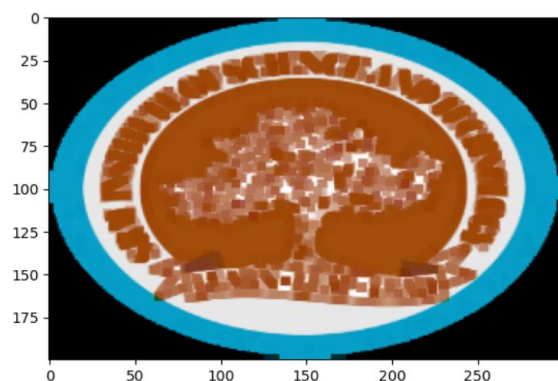
```
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

```
i=cv2.imread('SRM.png')
img=cv2.resize(i,(300,200))
plt.imshow(img)
```

Input Image:

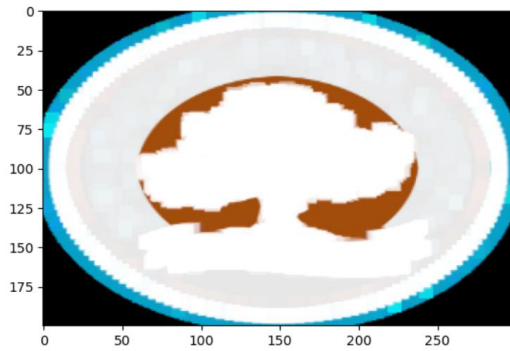


```
kernel = np.ones((5,5), np.uint8)
img_erosion = cv2.erode(img, kernel, iterations=1)
plt.imshow(img_erosion)
```



```
img_dilation = cv2.dilate(img, kernel, iterations=2)  
plt.imshow( img_dilation)
```

Output:-



Result : Erosion and dilation of images successfully

Experiment-8

Experiment : Write a program of computer vision to detect multiple faces.

Aim: computer vision program to detect multiple faces.

Procedure:

1. Start Jupyter Notebook
2. Import CV2
3. Insert image and read image
4. Convert image to grayscale
5. Detect faces in image
6. Draw rectangle around the image
7. Show image
8. End program

Program/ Code :

Import cv2

Load the pre-trained face detection classifier

```
face_cascade = cv2.CascadeClassifier(cv2.data.haarcascades + "haarcascade_frontalface_default.xml")
```

Read the image

```
img = cv2.imread("h1.jpg")
```

Convert the image to grayscale

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

Detect faces in the image

```
faces = face_cascade.detectMultiScale(gray, scaleFactor=1.1, minNeighbors=5, minSize=(50, 50))
```

Draw rectangles around the detected faces

for (x, y, w, h) in faces:

```
    cv2.rectangle(img, (x, y), (x + w, y + h), (255, 0, 0), 2)
```

Display the image with detected faces

```
cv2.imshow("Faces", img)
```

```
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```

Input / Output:

Input



Output



Result: Face detected successfully.

Experiment-9

Experiment: Write a program in computer vision using tesseract for text detection and print it.

Aim: Using tesseract library to detect text from image and print it.

Procedure:

1. Start Jupyter Notebook
2. Import CV2
3. Import pytesseract
4. Import pytesseract exe from C drive
5. Import image
6. Convert image to grayscale
7. Apply thresholding to preprocess the image
8. Perform OCR
9. Print text
10. End program

Program/ Code :

```
import cv2
import pytesseract

# Install tesseract-ocr and add it to path
pytesseract.pytesseract.tesseract_cmd = 'C:/Program Files/Tesseract-OCR/tesseract.exe'

# Read the image
img = cv2.imread('imm.png')

# Convert to grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Apply thresholding to preprocess the image
thresh = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)[1]

# Perform OCR
text = pytesseract.image_to_string(thresh)

#Print detected text
print(text)
```

Input

The quick brown dog jumped over the lazy fox. The quick brown dog jumped over the lazy fox. The quick brown dog jumped over the lazy fox. The quick brown dog jumped over the lazy fox.

Output:

The quick brown dog jumped over the lazy fox. The quick brown dog jumped over the lazy fox. The quick brown dog jumped over the lazy fox. The quick brown dog jumped over the lazy fox.

Result:

Text Detected Successfully.

Experiment 10

Experiment: Write a program in computer vision for real time object detection from a video using YOLO

Aim: using YOLO(You Look Only) code , detect an object .

Procedure:

1. Step 1: Setup the Environment.
2. Step 2: Importing Necessary Libraries.
3. Step 3: Define Function to Get Class Colours .
4. Step 4: Load YOLO Model.
5. Step 5: Open Video Capture.
6. Step 6: Process Video Frames.
7. Step 7: Main Loop for Real-time Object Detection.
8. Step 8: Display Image and Break Loop.

Program/ Code :

```
import cv2
from yolo_predictions import YOLO_Pred

#load dataset
yolo = YOLO_Pred('./Model/weights/best.onnx','data.yaml')

#load image

img = cv2.imread('./street_image.jpg')

#show image
cv2.imshow('img',img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```



```
# predictions  
img_pred = yolo.predictions(img)
```

Output predictions



Result: Object detected from input image successfully

Experiment 11

Experiment: Write a program in computer vision for using gushing blur function.

Aim: Using YOLO(You Look Only) code , detect an object .

Procedure:

1. Import the OpenCV library.
2. Read the input image using cv2.imread().
3. Apply Gaussian Blur using a **5×5 kernel** with cv2.GaussianBlur().
4. Display the original and blurred images.
5. Wait for a key press and close all windows.

Code:

```
import cv2
image = cv2.imread('sample.jpg')

if image is not None:
    # Apply Gaussian Blur with a 5x5 kernel
    gaussian_blur = cv2.GaussianBlur(image, (5, 5), 0) #Applied function

    # To create more blur, you can increase the ksize (kernel size) or the sigmaX parameter in
    cv2.GaussianBlur.

    cv2.imshow('Original', image)
    cv2.imshow('Gaussian Blur', gaussian_blur)

    cv2.waitKey(0)
    cv2.destroyAllWindows()
```

Input / Output:

Original image



blur image



Result: Image blurred by using Gaussian Blur function successfully.

Experiment - 12

Experiment: Write a Python program using OpenCV to convert an input image into grayscale and apply the Laplacian filter to detect edges. Display both the original image and the resulting Laplacian image.

Procedure:

1. Read the input image.
2. Convert the image to grayscale.
3. Apply the **Laplacian filter**.
4. Convert the result for display.
5. Display the original and Laplacian images.

Code:

```
import cv2
from google.colab.patches import cv2_imshow
img = cv2.imread("/content/srmlogo.jpg")

if img is None:
    print("Image not found!")
else:
    # Convert image to grayscale
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    # Apply Laplacian filter
    laplacian = cv2.Laplacian(gray, cv2.CV_64F)

    # Convert result for display
    laplacian = cv2.convertScaleAbs(laplacian)

print("Original Image:")      # Display images
cv2_imshow(img)

print("Laplacian Image:")
cv2_imshow(laplacian)
```

Input/ output:



Result: Edges Detected Successfully by using Laplacian Method.

Experiment- 13

Experiment: Write a Python program using OpenCV to increase and decrease the brightness of an input image. Display the original, brightened, and darkened image(**Image Segmentation**).

Procedure:

1. Read the input image using OpenCV.
2. Increase brightness using `cv2.convertScaleAbs()`.
3. Decrease brightness using `cv2.convertScaleAbs()`.
4. Display the original, brightened, and darkened images.

Code:

```
import cv2
import numpy as np
from google.colab.patches import cv2_imshow

# Load image
img = cv2.imread("srm.png")

# Increase brightness
bright = cv2.convertScaleAbs(img, alpha=0.5, beta=60)

# Decrease brightness
dark = cv2.convertScaleAbs(img, alpha=1.2, beta=-40)

cv2_imshow(img)
print("Original Image")
cv2_imshow(bright)
print("Bright Image")
cv2_imshow(dark)
print("Dark Image")
```

Input/ Output:



Original Image



Bright Image



Dark Image

Result: Image segmentation program executed successfully.

Experiment: 14

Experiment 15: Write a program in computer vision for live streaming or opening a camera.

Aim: Program for opening a camera using CV code.

Procedure:

1. Step 1: Setup the Environment.
2. Step 2: Importing Necessary Libraries.
3. Step 3: code for live camera Video Capture object.
4. Step 4: Check if camera opened successfully or not.
5. Step 5: Check that frame is reading correctly
6. Step 6: display the camera or resultant frame
7. Step 7: display the camera or resultant frame
8. Step 8: press C for close the camera

Program/ Code :

```
import cv2

# code for live camera Video Capture object
cap = cv2.VideoCapture(0)          # Here 0 means default camera

# Check if camera opened successfully or not
if not cap.isOpened():
    print("Error opening video stream or file")

while True:
    # Read a frame from the camera
    ret, frame = cap.read()

    # Check that frame is reading correctly
    if not ret:
        print("Can't receive frame (stream end?). Exiting ...")
        break

    # display the camera or resultant frame
    cv2.imshow('Live Camera', frame)

    # press c for close the camera

    if cv2.waitKey(1) == ord('c'):
        break

# Release the camera and close all windows
cap.release()
cv2.destroyAllWindows()
```

Output: My camera image .

Experiment- 15

Experiment : Write a program in computer vision for contour detection and masking.

Aim: Using opencv application in computer vision for contour detection and masking

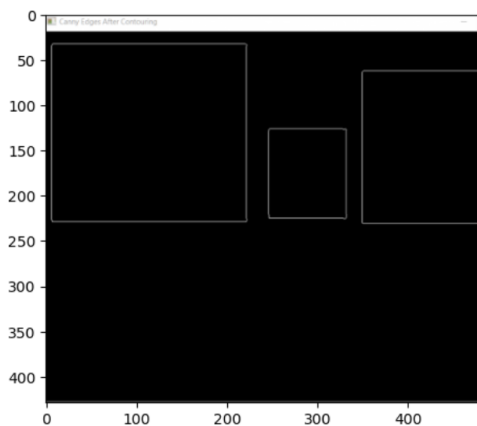
Procedure:

1. Start Jupyter Notebook
2. Import CV2
3. Import pytesseract
4. Import pytesseract exe from C drive
5. Import image
6. Convert image to grayscale
7. Apply thresholding to preprocess the image
8. Perform OCR
9. Print text
10. End program

Program/ Code :

```
import cv2
import matplotlib.pyplot as plt
import numpy as np
```

```
img=cv2.imread('111.png')
plt.imshow(img)
```



```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

```
# Find Canny edges
```

```
edged = cv2.Canny(gray, 30, 200)
```

```
cv2.waitKey(0)
```

```
# Finding Contours
```

```
contours, hierarchy = cv2.findContours(edged, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_NONE)
```

```
#cv2.imshow('Canny Edges After Contouring', edged)
```

```
#cv2.waitKey(0)
```

```
#print("Number of Contours found = " + str(len(contours)))
```

Draw all contours

-1 signifies drawing all contours

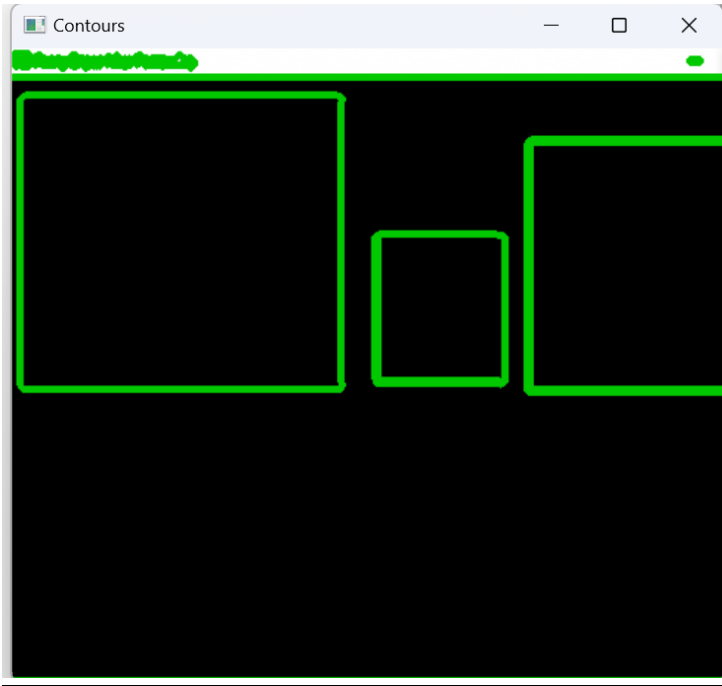
```
cv2.drawContours(img, contours, -1, (0, 200, 2), 3)
```

```
cv2.imshow('Contours', img)
```

```
cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```

Output:



Result: Represent the boundaries of the object(contour) successfully.